BUT Informatique 3e année

Examen de TP Billetterie de cinéma professionnelle

Java 25 Maven Spring Boot MySQL

Élément	Cadre proposé
Durée	4 h
Modalité	Travail individuel
Livrable	Dépôt Maven complet et exécutable
Commande de référence	.\mvnw.cmd clean verify sous Windows ou ./mvnw clean verify
Évaluation	20 points

Nom et prénom _____

Groupe et poste _____

Mission

Vous devez terminer le backend d'une billetterie de cinéma. Le service publie les films et les séances, protège les opérations d'administration, accepte des réservations et garantit qu'une séance ne dépasse jamais la capacité de sa salle. Le résultat doit fonctionner hors de l'IDE et se vérifier par une construction Maven reproductible.

Résultat attendu

À la fin de l'épreuve, le projet démarre avec une base MySQL neuve, applique ses migrations, expose une API documentée, refuse les données invalides et les accès non autorisés, résiste à deux réservations concurrentes et passe les tests automatisés.

Cadre de travail

Le dépôt initial contient le Maven Wrapper, un projet Spring Boot incomplet, un fichier Docker Compose pour MySQL et quelques classes ou tests à compléter. Vous pouvez réorganiser le code sans modifier les contrats de recette indiqués dans ce sujet.

Contraintes techniques

- Java 25 pour la compilation, les tests et l'exécution.
- Spring Boot 4.1.1 avec le parent Maven Spring Boot.
- Maven Wrapper versionné dans le dépôt. Le build IntelliJ ne constitue pas une preuve de conformité.
- MySQL comme base de référence. H2 n'est pas utilisé pour masquer des différences de dialecte ou de contraintes.
- API REST Spring MVC, persistance Spring Data JPA, migrations Flyway, validation Jakarta, Spring Security et Actuator.
- DTO dédiés aux entrées et sorties. Les entités JPA ne sont pas renvoyées directement par les contrôleurs.
- Les versions gérées par le parent Spring Boot ne sont pas répétées dans chaque dépendance.

Dépendances à mobiliser

Le tableau distingue le socle applicatif, les éléments nécessaires à une exploitation professionnelle et les outils de test. Une dépendance n'est créditée que si elle répond à un besoin effectivement implémenté.

Niveau	Dépendances	Usage évalué
Socle	webmvc, data-jpa, validation, mysql-connector-j	HTTP et JSON, modèle JPA, contrôle des entrées, accès MySQL
Professionnel	security, flyway, flyway-mysql, actuator	Autorisations, schéma versionné, contrôle de santé
Documentation	springdoc-openapi-starter-webmvc-ui 3.1.1	Contrat OpenAPI et interface Swagger UI
Tests	starter-test et starters de test ciblés	Tests unitaires, MVC, JPA et sécurité
Intégration	spring-boot-testcontainers et Testcontainers MySQL	Tests sur le même moteur que l'application

Lombok et MapStruct sont facultatifs.

La solution attendue peut être écrite sans processeur d'annotations. Si vous en ajoutez un, sa configuration Maven doit être explicite et compatible avec Java 25. Aucun point n'est attribué pour son seul ajout.

Éléments hors périmètre

Aucune interface graphique, aucun paiement réel, aucun envoi d'e-mail et aucune inscription d'utilisateur ne sont demandés. Les comptes de démonstration peuvent être configurés en mémoire pour concentrer l'épreuve sur les autorisations.

Modèle métier

Le modèle minimal comprend quatre agrégats. Les noms Java peuvent être en français ou en anglais, mais doivent rester cohérents dans le code et dans l'API.

Entité	Données minimales	Règles
Film	id, titre, genre, dureeMinutes, dateSortie	Titre obligatoire, durée comprise entre 40 et 300 minutes
Salle	id, nom, capacite	Nom unique, capacité comprise entre 20 et 500
Séance	id, film, salle, debut, prixUnitaire	Film et salle obligatoires, prix strictement positif
Réservation	id, code, seance, emailClient, nombrePlaces, statut, creeeLe	1 à 8 places, code unique, statut CONFIRMEE ou ANNULEE

Invariant de capacité

Pour une séance, la somme des places des réservations confirmées ne doit jamais dépasser la capacité de la salle. Une annulation libère les places. La vérification doit rester correcte lorsque deux requêtes arrivent en même temps sur deux threads ou deux instances de l'application.

Une synchronisation seulement en mémoire n'est pas recevable.

Vous devez utiliser une transaction et une stratégie explicite de concurrence fondée sur la base, par exemple un verrou pessimiste ou un verrou optimiste avec gestion du conflit. Justifiez le choix dans le README.

Contrat HTTP minimal

Méthode et chemin	Accès	Résultat principal
GET /api/films	Public	200 et liste filtrable par titre ou genre
GET /api/seances?du=&au=	Public	200 et séances dans l'intervalle
POST /api/programmation/films	PROGRAMMATEUR ou ADMIN	201 et ressource créée
POST /api/programmation/seances	PROGRAMMATEUR ou ADMIN	201 ou 400
POST /api/admin/salles	ADMIN	201 ou 400 ou 409
POST /api/reservations	CLIENT	201 ou 400 ou 404 ou 409
DELETE /api/reservations/{code}	CLIENT ou ADMIN	204 ou 404
GET /actuator/health	Public	200 et état de l'application

Les lectures publiques ne doivent pas exposer de données techniques inutiles. Les créations renvoient une réponse cohérente et, si possible, un en-tête Location.

Travail demandé

1 Construire un projet reproductible

Complétez le fichier pom.xml et vérifiez l'environnement utilisé par Maven. Le projet doit déclarer Java 25 une seule fois et ne doit pas combiner des valeurs contradictoires de source, target et release.

```
.\mvnw.cmd -version
.\mvnw.cmd clean verify
```

- Conserver le parent Spring Boot et sa gestion de versions.
- Déclarer les scopes runtime et test correctement.
- Conserver le spring-boot-maven-plugin dans la section build et non dans dependencies.
- Documenter dans le README le JDK attendu et la commande de vérification.

2 Créer et contrôler le schéma

Créez une migration Flyway V1 qui définit les tables, clés étrangères, index et contraintes nécessaires. Ajoutez quelques films, une salle et une séance de démonstration dans une migration ou un jeu de données distinct.

```
spring.jpa.hibernate.ddl-auto=validate
```

- Aucune création automatique du schéma par Hibernate n'est acceptée.
- La base doit pouvoir être supprimée puis reconstruite uniquement par les migrations.
- Le prix utilise BigDecimal côté Java et un type DECIMAL adapté côté SQL.
- Le code de réservation et le nom de salle disposent d'une contrainte d'unicité.

3 Implémenter la couche REST

Implémentez les endpoints minimaux avec des contrôleurs fins, des services responsables des règles métier, des repositories Spring Data et des DTO immuables. Les entrées sont validées avant l'appel métier.

```
POST /api/reservations
{
  "seanceId": 12,
  "emailClient": "lea@example.test",
  "nombrePlaces": 3
}
```

La réponse de création contient au minimum le code de réservation, le statut, le nombre de places et le montant total calculé côté serveur.

Validation et erreurs

Centralisez les erreurs avec un ControllerAdvice. Les messages restent compréhensibles, sans trace de pile ni détail SQL. Les erreurs produisent un corps JSON stable.

```
{
  "timestamp": "2026-10-17T09:12:00Z",
  "status": 409,
  "code": "CAPACITE_INSUFFISANTE",
  "message": "Il ne reste que 2 places pour cette séance",
  "path": "/api/reservations"
}
```

Situation	Statut attendu	Exemple de code
Données invalides	400	VALIDATION_ECHOUEE
Ressource absente	404	SEANCE_INTROUVABLE
Capacité insuffisante ou doublon	409	CAPACITE_INSUFFISANTE
Authentification absente	401	Réponse Spring Security
Rôle insuffisant	403	Réponse Spring Security

4 Protéger les opérations

Configurez Spring Security avec authentification HTTP Basic pour l'épreuve. Les mots de passe ne sont jamais stockés en clair dans le code source. Utilisez BCrypt et fournissez trois comptes de démonstration documentés dans le README.

Rôle	Droits
CLIENT	Créer et annuler une réservation
PROGRAMMATEUR	Créer des films et des séances
ADMIN	Créer des salles et accéder à toutes les opérations protégées

Les routes de lecture, la documentation OpenAPI et /actuator/health restent publiques. Les autres endpoints Actuator ne sont pas exposés publiquement.

5 Garantir la capacité sous concurrence

Placez la règle de réservation dans une transaction. Le calcul des places restantes et l'enregistrement de la réservation forment une seule opération cohérente. En cas de conflit, l'API répond 409 et n'enregistre aucune réservation partielle.

Scénario obligatoire.

Une séance possède 5 places restantes. Deux requêtes concurrentes demandent chacune 4 places. Une seule doit réussir et le nombre final de places réservées ne doit pas dépasser la capacité.

6 Vérifier le comportement

Écrivez des tests ciblés. La quantité compte moins que la couverture des risques du sujet. Les tests doivent être lisibles, déterministes et exécutés par Maven.

Type de test	Couverture minimale
Service unitaire	Calcul du montant, capacité insuffisante et annulation
MVC	Création valide, validation 400, authentification 401 et rôle 403
JPA	Requête de recherche ou de somme des places confirmées
Intégration MySQL	Migration, réservation réelle et contrainte ou transaction
Concurrence	Deux réservations simultanées sur la même séance

Le test MySQL utilise Testcontainers. S'il est impossible de lancer Docker sur le poste, conservez le test et expliquez clairement la limitation dans le README ; les autres tests doivent continuer à s'exécuter.

7 Rendre le service exploitable

- Exposer /actuator/health et limiter l'exposition des autres endpoints de gestion.
- Rendre la description OpenAPI sur /v3/api-docs et Swagger UI sur /swagger-ui.html.
- Ajouter un README court avec prérequis, démarrage MySQL, lancement, comptes de test, commandes Maven, endpoints et décision de concurrence.
- Vérifier le JAR hors IntelliJ avec java -jar après la construction Maven.

```
.\mvnw.cmd clean verify
java -jar target\nom-du-projet.jar
```

Livrables

- Code source et tests dans une arborescence Maven standard.
- pom.xml, mvnw, mvnw.cmd et répertoire .mvn.
- Migrations Flyway versionnées.
- Configuration sans secret réel ni mot de passe de production.
- README.md permettant à une autre personne de lancer et vérifier le projet.

Ne remettez pas les répertoires target, les fichiers de configuration de l'IDE, les journaux ni les secrets locaux.

Recette fonctionnelle

Les contrôles suivants seront exécutés sur une copie propre du dépôt. Ils servent de définition de fin de travail.

ID	Contrôle	Résultat attendu
A1	Environnement	Le Maven Wrapper s'exécute avec Java 25 et clean verify termine avec succès.
A2	Base neuve	Sur une base vide, Flyway applique les migrations et Hibernate valide le schéma.
A3	Lecture publique	Un appel anonyme obtient les films et les séances avec un JSON maîtrisé.
A4	Validation	Une réservation à 0 ou 9 places reçoit 400 avec un corps d'erreur stable.
A5	Sécurité	CLIENT ne crée pas de salle ; ADMIN le peut ; un appel protégé anonyme reçoit 401.
A6	Réservation	Une réservation valide reçoit 201 avec code, statut, quantité et montant total.
A7	Capacité	Une demande supérieure aux places restantes reçoit 409 sans écriture partielle.
A8	Concurrence	Deux demandes simultanées ne provoquent aucune surréservation.
A9	Annulation	Une annulation répond 204 et libère les places pour une réservation suivante.
A10	Exploitation	Actuator Health et la documentation OpenAPI sont accessibles aux chemins prévus.

Règles de qualité

- Les contrôleurs ne contiennent pas la logique de capacité.
- Les services délimitent les transactions et nomment clairement les règles métier.
- Les exceptions métier sont spécifiques et traduites au bon statut HTTP.
- Les dépendances inutilisées ou ajoutées sans justification peuvent être retirées lors de la correction.
- Le résultat de clean verify prime sur l'état affiché par l'IDE.

Conseils de gestion du temps

Temps indicatif	Priorité
25 min	Environnement Maven, dépendances et démarrage MySQL
50 min	Migration et modèle JPA
75 min	API, DTO, validation et erreurs
40 min	Sécurité et rôles
30 min	Transaction et concurrence
20 min	Tests, documentation et vérification finale

Barème sur 20

Critère	Points	Éléments observés
Environnement et Maven	1,5	Java 25 cohérent, Wrapper, scopes et build reproductible
Persistance et migrations	3,5	Modèle, relations, contraintes, Flyway et ddl-auto validate
API et qualité du contrat	4,5	Endpoints, DTO, validation, erreurs, statuts et montant calculé
Sécurité	2,5	Authentification, BCrypt, rôles, routes publiques et protégées
Capacité et concurrence	3,5	Transaction, verrouillage cohérent, conflit 409 et absence de surréservation
Tests	3,0	Service, MVC, JPA, sécurité, MySQL et concurrence
Exploitation et documentation	1,5	Actuator, OpenAPI, README et démarrage hors IDE
Total	20,0	

Un projet qui ne compile pas avec le Maven Wrapper ne permet pas d'exécuter la recette automatisée. Le code reste examiné, mais les points liés aux comportements non vérifiables ne peuvent pas être attribués.

Bonus plafonné

Jusqu'à 1 point peut compenser une faiblesse ailleurs sans dépasser 20 sur 20. Le bonus doit être fonctionnel et testé.

- ETag ou contrôle de version HTTP cohérent sur une ressource modifiable.
- Idempotence de la création de réservation avec une clé fournie par le client.
- Métrique métier Micrometer sur le nombre de places réservées et refusées.

Aide autorisée

La documentation officielle de Java, Maven, Spring Boot, Spring Security, Flyway, Testcontainers et springdoc-openapi peut être consultée. L'utilisation d'un exemple externe ne dispense pas d'expliquer les choix du projet ni de respecter les versions imposées.

Points de vigilance avec Java 25

- Comparer le JDK du projet, le JDK qui exécute Maven et le runtime utilisé pour lancer le JAR.
- Utiliser release 25 au lieu d'empiler des valeurs source et target contradictoires.
- Avec un processeur d'annotations, déclarer explicitement les processeurs autorisés. Un ancien Lombok peut échouer avec Java 25.
- Après une modification des processeurs, lancer clean verify afin d'éliminer les sources générées obsolètes.

Annexe des dépendances Maven

Le parent Spring Boot 4.1.1 gère les versions du socle Spring et de nombreuses dépendances associées. Sauf mention contraire, n'ajoutez pas de version explicite aux éléments gérés par ce parent.

Coordonnées	Scope	Rôle
org.springframework.boot:spring-boot-starter-webmvc	compile	API Spring MVC et serveur embarqué
org.springframework.boot:spring-boot-starter-data-jpa	compile	Entités, repositories et transactions
org.springframework.boot:spring-boot-starter-validation	compile	Validation Jakarta des DTO
org.springframework.boot:spring-boot-starter-security	compile	Authentification et autorisations
org.springframework.boot:spring-boot-starter-flyway	compile	Intégration des migrations
org.flywaydb:flyway-mysql	runtime	Support MySQL pour Flyway
com.mysql:mysql-connector-j	runtime	Pilote JDBC MySQL
org.springframework.boot:spring-boot-starter-actuator	compile	Santé et exploitation
org.springdoc:springdoc-openapi-starter-webmvc-ui:3.1.1	compile	OpenAPI et Swagger UI
org.springframework.boot:spring-boot-starter-test	test	JUnit Jupiter, AssertJ, Mockito et socle de test
org.springframework.boot:spring-boot-starter-webmvc-test	test	Tests MVC ciblés
org.springframework.boot:spring-boot-starter-data-jpa-test	test	Tests JPA ciblés
org.springframework.boot:spring-boot-starter-security-test	test	Support des tests de sécurité
org.springframework.boot:spring-boot-testcontainers	test	ServiceConnection et intégration Spring Boot
org.testcontainers:testcontainers-mysql	test	Conteneur MySQL
org.testcontainers:testcontainers-junit-jupiter	test	Cycle de vie JUnit Jupiter

Dépendances à ne pas ajouter par réflexe

Redis, Kafka, RabbitMQ, un SDK de paiement, un client de messagerie ou un mécanisme JWT ne répondent à aucun besoin du périmètre minimal. Leur présence n'améliore pas la note si la fonctionnalité correspondante n'existe pas et n'est pas testée.

Références techniques

Compatibilité Spring Boot https://docs.spring.io/spring-boot/system-requirements.html
Starters et gestion des dépendances https://docs.spring.io/spring-boot/reference/using/build-systems.html
Processeurs d'annotations Maven https://maven.apache.org/plugins/maven-compiler-plugin-4.x/examples/annotation-processor.html
Testcontainers avec Spring Boot https://docs.spring.io/spring-boot/reference/testing/testcontainers.html
springdoc-openapi https://springdoc.org/